

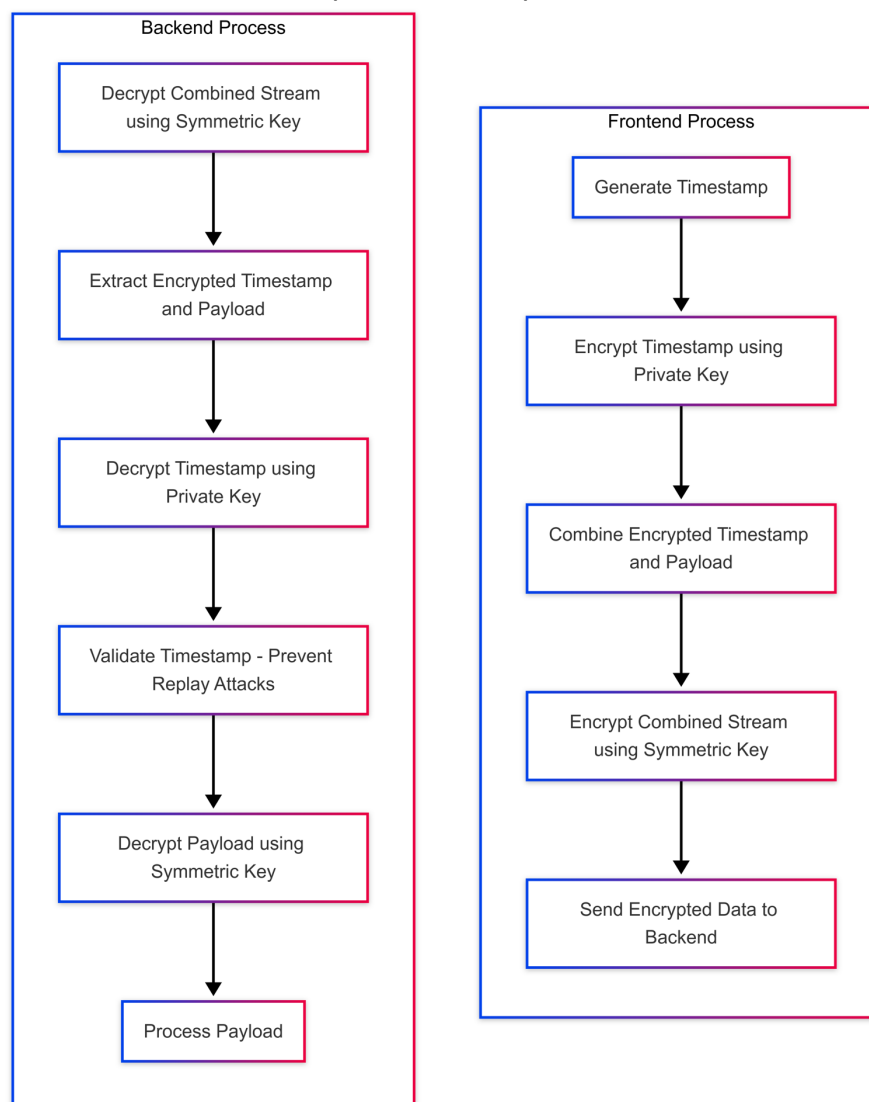
Algorithm for Secure Web Interface with Embedded Timestamp Encryption

Introduction

Implementing an algorithm to embed encrypted timestamps into web requests enhances security by ensuring that each request carries a unique and verifiable timestamp. This practice helps prevent replay attacks and ensures the integrity and confidentiality of the communication between the client and server.

Objective

The goal of using encrypted timestamps is to ensure dynamic, secure key generation and payload encryption. By embedding timestamps in web requests, we can create a safe mechanism that guarantees the authenticity and freshness of each request, making it difficult for attackers to replicate or tamper with the data.



Frontend Implementation

Steps Involved

- Generate a timestamp.
- Encrypt the timestamp using a symmetric key.
- Combine the encrypted timestamp with the payload data.
- Perform symmetric encryption on the combined data.

Node.js Code Example

```
const crypto = require('crypto');
function generateTimestamp() {
    return new Date().toISOString();
}
function encryptTimestamp(timestamp, key) {
    const cipher = crypto.createCipher('aes-256-cbc', key);
    let encrypted = cipher.update(timestamp, 'utf8', 'hex');
    encrypted += cipher.final('hex');
    return encrypted;
}
function combineAndEncryptPayload(payload, encryptedTimestamp, key) {
    const combinedData = JSON.stringify({ payload, encryptedTimestamp });
    const cipher = crypto.createCipher('aes-256-cbc', key);
    let encrypted = cipher.update(combinedData, 'utf8', 'hex');
    encrypted += cipher.final('hex');
    return encrypted;
}
// Example usage
const key = 'mysecretkey';
const timestamp = generateTimestamp();
const encryptedTimestamp = encryptTimestamp(timestamp, key);
const payload = { data: 'example payload' };
const encryptedPayload = combineAndEncryptPayload(payload, encryptedTimestamp, key);
console.log('Encrypted Payload:', encryptedPayload);
```

Backend Implementation

Steps to Decrypt and Validate

- Decrypt the combined payload and timestamp using the symmetric key.
- Extract and validate the timestamp.
- Process the payload data.

Node.js Code Example

```
const crypto = require('crypto');
function decryptCombinedData(encryptedData, key) {
    const decipher = crypto.createDecipher('aes-256-cbc',
key);
    let decrypted = decipher.update(encryptedData, 'hex',
'utf8');
    decrypted += decipher.final('utf8');
    return JSON.parse(decrypted);
}
function validateTimestamp(timestamp) {
    const now = new Date();
    const requestTime = new Date(timestamp);
    const diff = Math.abs(now - requestTime);
    const threshold = 5 * 60 * 1000; // 5 minutes
    return diff < threshold;
}
// Example usage
const key = 'mysecretkey';
const encryptedPayload = '...'; // Received encrypted payload
const decryptedData = decryptCombinedData(encryptedPayload,
key);
if (validateTimestamp(decryptedData.encryptedTimestamp)) {
    console.log('Valid timestamp:',
decryptedData.encryptedTimestamp);
    console.log('Payload:', decryptedData.payload);
} else {
    console.log('Invalid timestamp');
}
```

JWT Tokens with Timestamp Encryption

Embedding Encrypted Timestamp into JWT Claims

- Generate and encrypt the timestamp.
- Embed the encrypted timestamp into the JWT claims.
- Create and sign the JWT token.

Node.js Code Example for Creating JWT Tokens

```
const jwt = require('jsonwebtoken');
const crypto = require('crypto');
function generateTimestamp() {
    return new Date().toISOString();
}
function encryptTimestamp(timestamp, key) {
    const cipher = crypto.createCipher('aes-256-cbc', key);
    let encrypted = cipher.update(timestamp, 'utf8', 'hex');
    encrypted += cipher.final('hex');
    return encrypted;
}
function createJwtToken(payload, key, secret) {
    const timestamp = generateTimestamp();
    const encryptedTimestamp = encryptTimestamp(timestamp,
key);
    const claims = { payload, timestamp: encryptedTimestamp };
    return jwt.sign(claims, secret);
}
// Example usage
const key = 'mysecretkey';
const secret = 'myjwtsecret';
const payload = { data: 'example payload' };
const token = createJwtToken(payload, key, secret);
console.log('JWT Token:', token);
```

Node.js Code Example for Validating JWT Tokens

```
const jwt = require('jsonwebtoken');
const crypto = require('crypto');

function decryptTimestamp(encryptedTimestamp, key) {
  const decipher = crypto.createDecipher('aes-256-cbc',
key);
  let decrypted = decipher.update(encryptedTimestamp, 'hex',
'utf8');
  decrypted += decipher.final('utf8');
  return decrypted;
}

function validateJwtToken(token, key, secret) {
  const decoded = jwt.verify(token, secret);
  const timestamp = decryptTimestamp(decoded.timestamp,
key);
  return validateTimestamp(timestamp) ? decoded.payload :
null;
}

function validateTimestamp(timestamp) {
  const now = new Date();
  const requestTime = new Date(timestamp);
  const diff = Math.abs(now - requestTime);
  const threshold = 5 * 60 * 1000; // 5 minutes
  return diff < threshold;
}

// Example usage
const key = 'mysecretkey';
const secret = 'myjwtsecret';
const token = '...'; // Received JWT token
const payload = validateJwtToken(token, key, secret);

if (payload) {
  console.log('Valid payload:', payload);
} else {
  console.log('Invalid timestamp in JWT token');
}
```

Security Considerations

- Prevent Replay Attacks: Ensure each timestamp is unique and validate its freshness.
- Secure Key Management: Use strong encryption keys and store them securely.
- Validate Timestamps: Implement a strict validation mechanism to check the timestamp's validity.